

SageTeck

Employee Evaluation Tasks (Web)

Foreword

The following task has been designed primarily to check for the required interest levels and skill set of the person applying for the job. Please note that it would be a big plus if the following task is completed within the due time. However, how you go about solving this task is important i.e. your problem tackling skills matter. It is recommended that you make a work log to show how you went about doing this task.

Task

Build a real-time analytics dashboard for an e-commerce platform called "TechMart". The system should process transaction data, detect anomalies, and provide actionable insights for business stakeholders.

Time Limit: 4-5 hours

Difficulty: Entry-level with progressive complexity

Target: Fresh full-stack developers

Dataset Description

You'll receive realistic e-commerce data with intentional complexity:

1. products.csv (500 products) - Tech products with categories, pricing, specifications
2. customers.csv (1000 customers) - Customer profiles with purchase history
3. transactions.csv (5000 transactions) - 30 days of transaction data
4. suppliers.csv (50 suppliers) - Supplier information with performance metrics

Core Requirements (Must Complete – 3-4 hours)

Part 1: Backend API Development (1.5 hours)

Database Schema & Setup (30 minutes)

Design and implement database schema for:

- Products (id, name, category, price, stock_quantity, supplier_id)
- Customers (id, email, registration_date, total_spent, risk_score)
- Transactions (id, customer_id, product_id, quantity, price, timestamp, status, payment_method)
- Suppliers (id, name, reliability_score, country)

REST API Endpoints (1 hour)

- GET /api/dashboard/overview - Summary statistics for last 24h
- POST /api/transactions - Create new transaction with validation
- GET /api/transactions/suspicious - Detect potentially fraudulent transactions
- GET /api/inventory/low-stock - Products below reorder threshold

- GET /api/analytics/hourly-sales - Hourly sales data for charts
- POST /api/alerts - Create custom business alerts

Data Validation & Business Logic:

- Transaction amounts must be reasonable (\$0.01 - \$10,000)
- Stock levels automatically update after purchases
- Calculate customer risk scores based on purchase patterns
- Implement basic fraud detection (unusual amounts, rapid purchases)

Part 2: Frontend Dashboard (1.5 hours)

Main Dashboard Layout (45 minutes)

- Header with real-time metrics (total sales, active users, alerts count)
- Grid layout with resizable widgets
- Sales performance chart (last 24 hours)
- Top products table with stock status indicators
- Recent transactions feed with risk indicators

Interactive Features (45 minutes)

- Real-time updates without page refresh
- Filtering by date range, product category, customer segment
- Sortable and searchable data tables
- Modal dialogs for transaction details
- Export functionality for reports (CSV/PDF)
- Responsive design for mobile/tablet

Complexity Challenges (Choose 1 – 2-3 hours)

Challenge A: Real-Time Fraud Detection System

Requirements:

- Implement streaming fraud detection using multiple algorithms
- Score transactions based on:
 - Velocity (multiple purchases in short time)
 - Amount anomalies (purchases significantly above customer average)
 - Geographic inconsistencies (IP location vs billing address)
 - Time patterns (purchases at unusual hours)

- Create real-time alerts with severity levels
- Dashboard widget showing fraud attempts with geographic visualization

Technical Complexity:

- WebSocket connections for real-time updates
- Statistical anomaly detection algorithms
- Rate limiting and sliding window calculations
- Geographic IP lookup integration

Challenge B: Advanced Inventory Management**Requirements:**

- Predictive stock level alerts using sales trend analysis
- Multi-supplier optimization (choose best supplier based on price, reliability, shipping time)
- Automated reorder suggestions with quantity optimization
- Seasonal demand pattern recognition
- Integration with external supplier API (mock service)

Technical Complexity:

- Time series analysis for demand forecasting
- Multi-criteria decision algorithms
- Background job processing for predictions
- External API integration with retry logic and fallbacks

Challenge C: Customer Behavior Analytics Engine**Requirements:**

- Customer segmentation based on purchase patterns (RFM analysis)
- Recommendation engine for cross-selling opportunities
- Customer lifetime value prediction
- Churn risk identification and prevention alerts
- Personalized dashboard views based on customer segments

Technical Complexity:

- Machine learning algorithms for clustering and prediction
- Complex SQL queries for behavioral analysis
- Caching strategies for expensive calculations
- A/B testing framework for recommendations

Challenge D: Performance Optimization & Scalability

Requirements:

- Handle 1000+ concurrent users with sub-200ms response times
- Database query optimization with proper indexing
- Implement caching layer (Redis) for frequently accessed data
- API rate limiting and request throttling
- Database connection pooling and query optimization
- Memory-efficient data processing for large datasets

Technical Complexity:

- Performance profiling and bottleneck identification
- Horizontal scaling considerations
- Efficient pagination for large datasets
- Memory leak prevention and monitoring

Deliverables

1. Working System

- Clear installation and setup instructions

2. Documentation

- README: System overview, setup instructions, usage examples
- Architecture Document: Key design decisions and trade-offs
- API Documentation: If you create an API interface

3. Evaluation Report

- Screenshots of dashboard
- Discussion of chosen complexity challenge and solution
- Identified limitations and potential improvements
- Performance characteristics and scalability considerations

4. Code Quality

- Clean, well-commented code
- Appropriate error handling and logging
- Unit tests for critical components
- Requirements.txt or equivalent dependency management

Recommended Tech Stack

Backend Technology Stack (Choose One)

- Node.js: Express.js, PostgreSQL/MySQL, Sequelize/Prisma ORM
- Python: FastAPI/Django, PostgreSQL/MySQL, SQLAlchemy/Django ORM
- Java: Spring Boot, PostgreSQL/MySQL, JPA/Hibernate
- C#: ASP.NET Core, SQL Server/PostgreSQL, Entity Framework

Frontend Technology Stack (Choose One)

- React: Create React App, Material-UI/Chakra UI, Chart.js/Recharts
- Vue.js: Vue CLI, Vuetify/Quasar, Chart.js/Vue-chartjs
- Angular: Angular CLI, Angular Material, Chart.js/ng2-charts

Required Features

- RESTful API with proper HTTP status codes
- Database migrations and seeders
- Input validation and error handling
- Unit tests for critical business logic
- API documentation (Swagger/OpenAPI)
- Dockerized deployment setup

Choose based on your comfort level

DEADLINES

The deadline for submitting your work log and source code is given in the Email. Please don't get discouraged if you run into problems, this task may be difficult for beginners so there are bound to be some confusions and issues.

Send your responses back to moeed@sageteck.com. Do contact us in case of any confusions. You are allowed to make any assumptions that you want, just please make sure that you document it in your work log.